

Linear Regression using Python



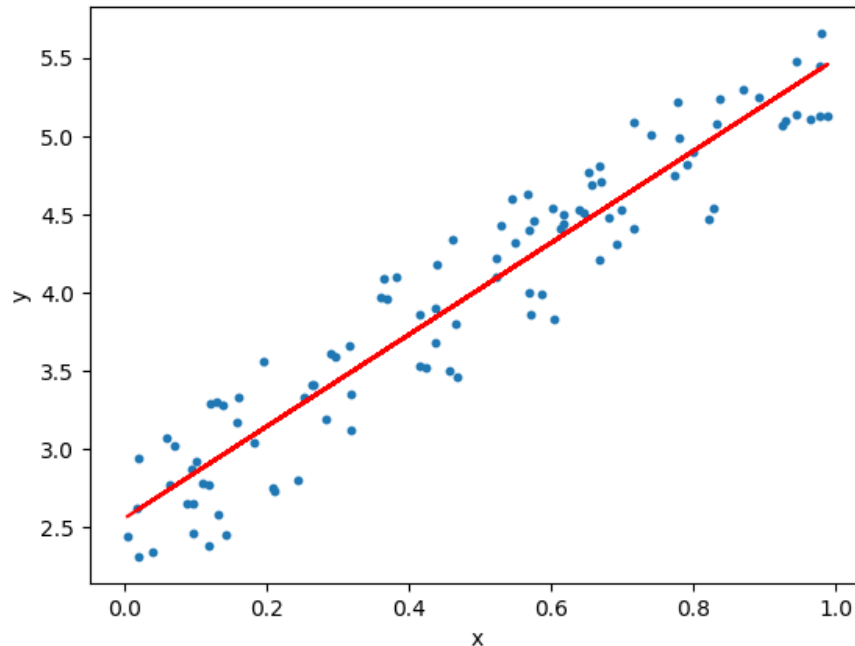
Animesh Agarwal [Follow](#)

Oct 5, 2018 · 7 min read

Linear Regression is usually the first machine learning algorithm that every data scientist comes across. It is a simple model but everyone needs to master it as it lays the foundation for other machine learning algorithms.

Where can Linear Regression be used?

It is a very powerful technique and can be used to understand the factors that influence profitability. It can be used to forecast sales in the coming months by analyzing the sales data for previous months. It can also be used to gain various insights about customer behaviour. By the end of the blog we will build a model which looks like the below picture i.e, determine a line which best fits the data.



This is the first blog of the machine learning series that I am going to cover. One can get overwhelmed by the number of articles in the web about machine learning algorithms. My purpose of writing this blog is two-fold. It can act as a guide to those who are entering into the field of machine learning and it can act as a reference for me.

Table of Contents

1. What is Linear Regression
2. Hypothesis of Linear Regression
3. Training a Linear Regression model
4. Evaluating the model
5. scikit-learn implementation

What is Linear Regression

The objective of a linear regression model is to find a relationship between one or more features(independent variables) and a continuous target variable(dependent variable). When there is only feature it is called *Uni-variate* Linear Regression and if there are multiple features, it is called *Multiple* Linear Regression.

Hypothesis of Linear Regression

The linear regression model can be represented by the following equation

$$Y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- Y is the predicted value
- θ_0 is the bias term.
- $\theta_1, \dots, \theta_n$ are the model parameters
- $x_1, x_2, \dots, x_n$ are the feature values.

The above hypothesis can also be represented by

$$Y = \theta^T x$$

where

- θ is the model's parameter vector including the bias term θ_0
- x is the feature vector with $x_0 = 1$

Data-set

Let's create some random data-set to train our model.

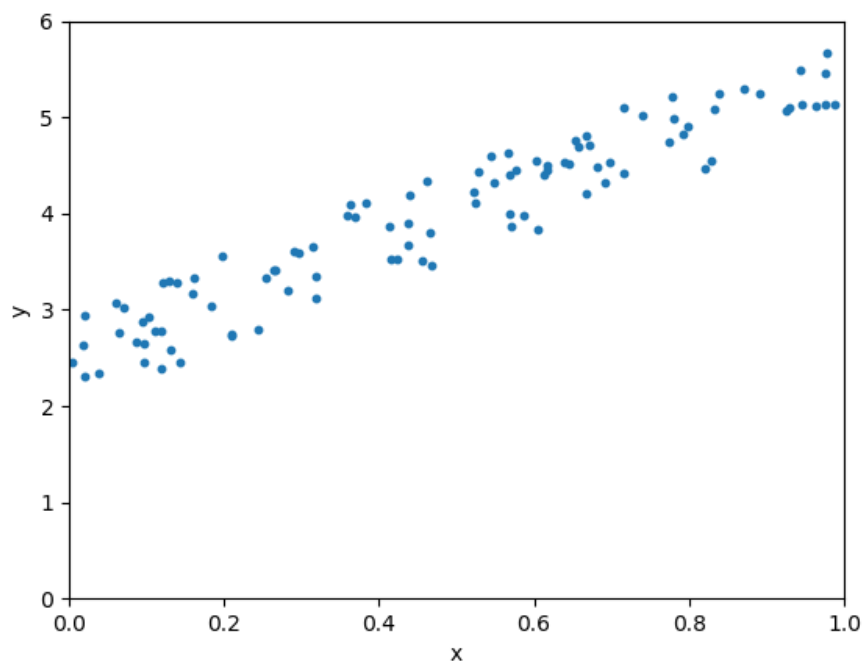
```
1 # imports
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # generate random data-set
6 np.random.seed(0)
7 x = np.random.rand(100, 1)
8 y = 2 + 3 * x + np.random.rand(100, 1)
9
```

```
10 # plot
11 plt.scatter(x,y,s=10)
12 plt.xlabel('x')
13 plt.ylabel('y')
14 plt.show()
```

RandomDataSet.py hosted with ❤ by GitHub

[view raw](#)

The plot for the data set generated using the above code is shown below:



Training a Linear Regression Model

Training of the model here means to find the parameters so that the model best fits the data.

How do we determine the best fit line?

The line for which the the *error* between the predicted values and the observed values is minimum is called the best fit line or the **regression** line. These errors are also called as **residuals**. The residuals can be visualized by the vertical lines from the observed data value to the regression line.



Image Credits: <http://wiki.engageeducation.org.au/further-maths/data-analysis/residuals/>

To define and measure the error of our model we define the cost function as the sum of the squares of the residuals. The cost function is denoted by

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h(x^i) - y^i)^2$$

where the hypothesis function $h(x)$ is denoted by

$$h(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

and m is the total number of training examples in our data-set.

Why do we take the square of the residuals and not the absolute value of the residuals ? *We want to penalize the points which are farther from the regression line much more than the points which lie close to the line.*

Our objective is to find the model parameters so that the cost function is minimum. We will use Gradient Descent to find this.

Gradient descent

Gradient descent is a generic optimization algorithm used in many machine learning algorithms. It iteratively tweaks the parameters of the model in order to minimize the cost function. The steps of gradient descent is outlined below.

1. We first initialize the model parameters with some random values. This is also called as **random initialization**.
2. Now we need to measure how the cost function changes with change in it's parameters. Therefore we compute the partial derivatives of the cost function w.r.t to the parameters $\theta_0, \theta_1, \dots, \theta_n$

$$\frac{\partial J(\theta)}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h(x^i) - y^i)$$

$$\frac{\partial J(\theta)}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h(x^i) - y^i) x_1^i$$

similarly, the partial derivative of the cost function w.r.t to any parameter can be denoted by

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h(x^i) - y^i) x_j^i$$

We can compute the partial derivatives for all parameters at once using

$$\begin{bmatrix} \frac{\partial J(\theta)}{\partial \theta_0} \\ \frac{\partial J(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{bmatrix} = \frac{1}{m} x^T (h(x) - y)$$

where $h(x)$ is

$$h(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

3. After computing the derivative we update the parameters as given below

$$\theta_0 = \theta_0 - \frac{\alpha}{m} \sum_{i=1}^m (h(x^i) - y^i)$$

$$\theta_1 = \theta_1 - \frac{\alpha}{m} \sum_{i=1}^m (h(x^i) - y^i) x_1^i$$

where α is the *learning parameter*.

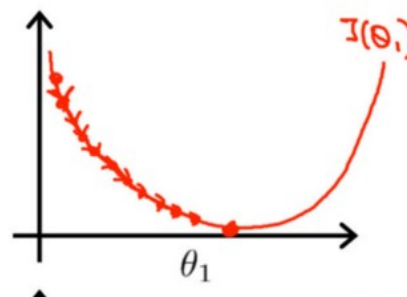
We can update all the parameters at once using,

$$\begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} - \alpha \begin{bmatrix} \frac{\partial J(\theta)}{\partial \theta_0} \\ \frac{\partial J(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{bmatrix}$$

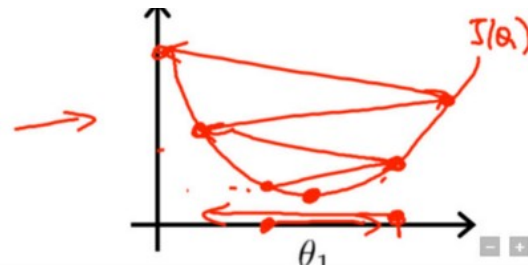
We repeat the steps 2,3 until the cost function converges to the minimum value. If the value of α is too small, the cost function takes larger time to converge. If α is too large, gradient descent may overshoot the minimum and may finally fail to converge.

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

If α is too small, gradient descent can be slow.



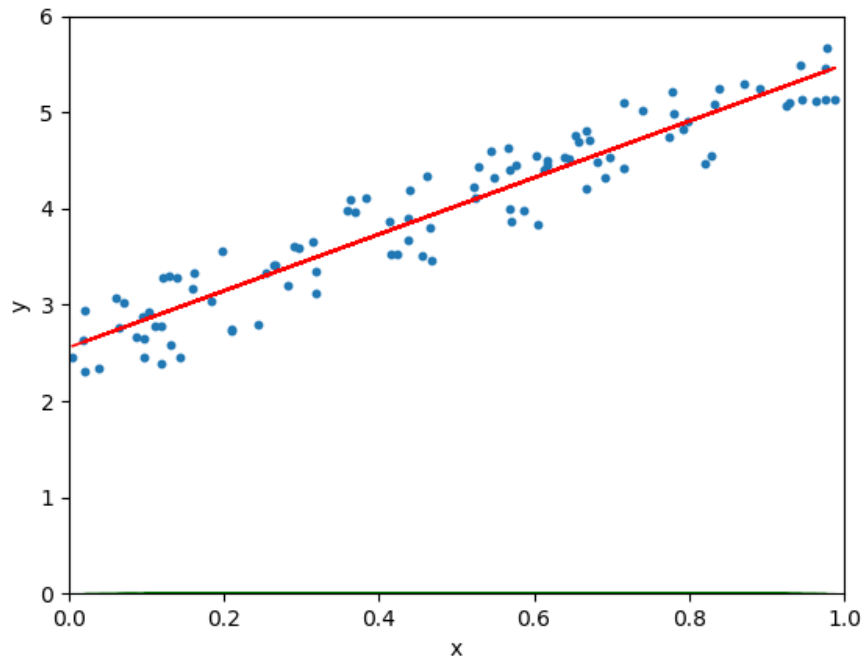
If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.



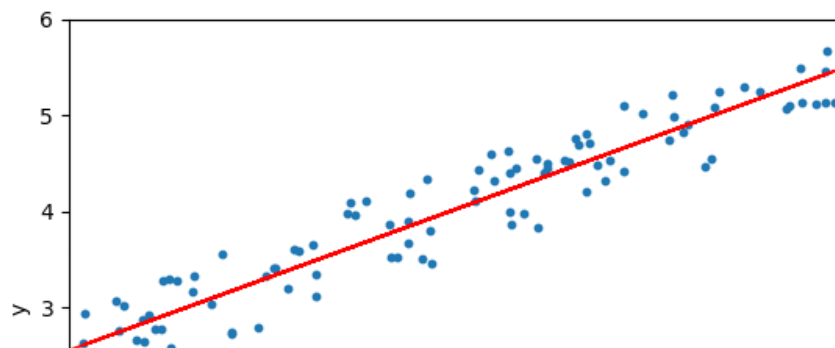
Source: Andrew Ng's course on Coursera

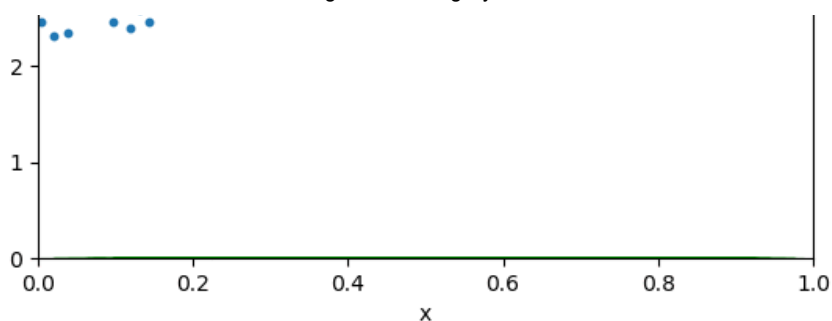
To demonstrate the gradient descent algorithm, we initialize the model parameters with 0. The equation becomes $Y = 0$. Gradient descent algorithm now tries to update the value of the parameters so that we arrive at the best fit line.

When the learning rate is very slow, the gradient descent takes larger time to find the best fit line.

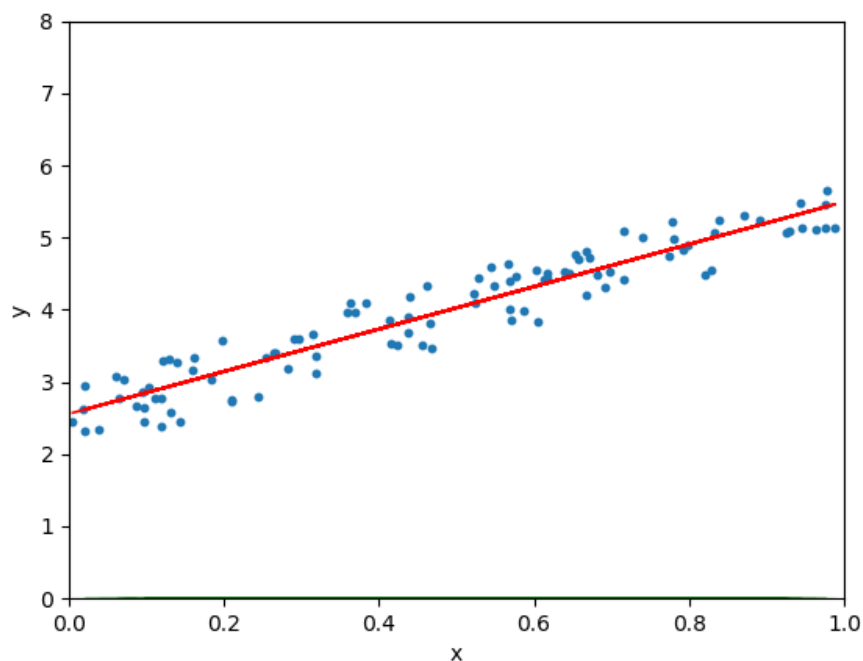


When the learning rate is normal





When the learning rate is arbitrarily high, gradient descent algorithm keeps overshooting the best fit line and may even fail to find the best line.



Implementing Linear Regression from scratch

The complete implementation of linear regression with gradient descent is given below.

```

1  # imports
2  import numpy as np
3
4
5  class LinearRegressionUsingGD:
6      """Linear Regression Using Gradient Descent.
7
8      Parameters
9      -----
10     eta : float
11         Learning rate
12     n_iterations : int
13         No of passes over the training set
14
15     Attributes

```

```

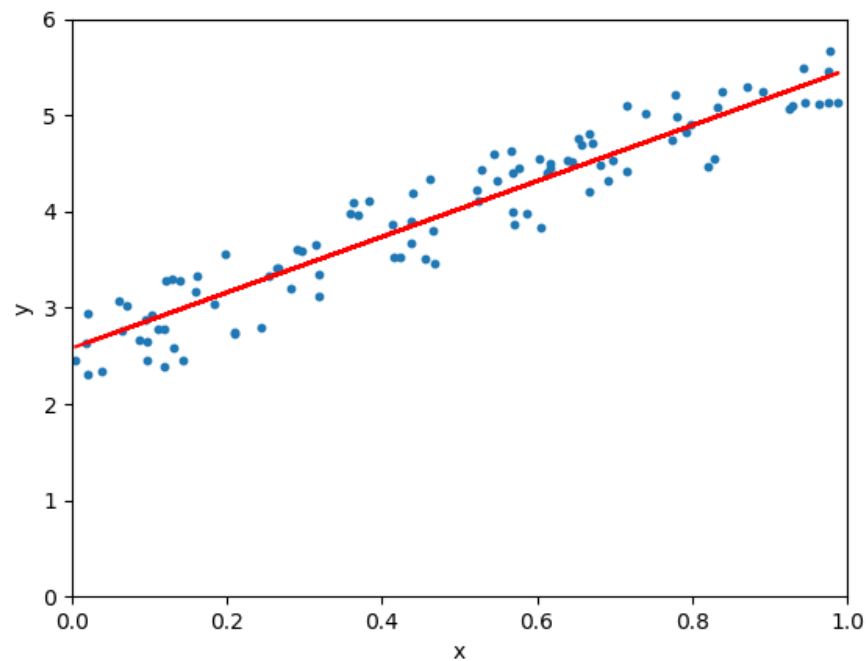
16         -----
17         w_ : weights/ after fitting the model
18         cost_ : total error of the model after each iteration
19
20         """
21
22         def __init__(self, eta=0.05, n_iterations=1000):
23             self.eta = eta
24             self.n_iterations = n_iterations
25
26         def fit(self, x, y):
27             """Fit the training data
28
29             Parameters
30             -----
31             x : array-like, shape = [n_samples, n_features]
32                 Training samples
33             y : array-like, shape = [n_samples, n_target_values]
34                 Target values
35
36             Returns
37             -----
38             self : object
39
40             """
41
42             self.cost_ = []
43             self.w_ = np.zeros((x.shape[1], 1))
44             m = x.shape[0]
45
46             for _ in range(self.n_iterations):
47                 y_pred = np.dot(x, self.w_)
48                 residuals = y_pred - y
49                 gradient_vector = np.dot(x.T, residuals)
50                 self.w_ -= (self.eta / m) * gradient_vector
51                 cost = np.sum((residuals ** 2)) / (2 * m)
52                 self.cost_.append(cost)
53             return self
54
55         def predict(self, x):
56             """ Predicts the value after the model has been trained.
57
58             Parameters
59             -----
60             x : array-like, shape = [n_samples, n_features]
61                 Test samples
62
63             Returns
64             -----
65             Predicted value
66
67             """
68             return np.dot(x, self.w_)

```

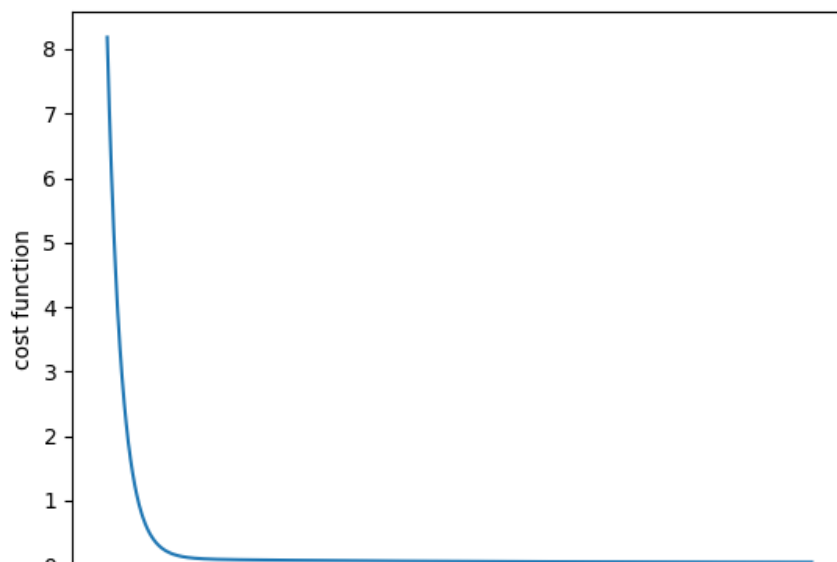

The model parameters are given below

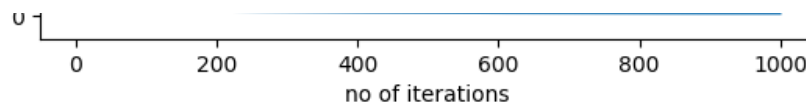
```
The coefficient is [2.89114079]  
The intercept is [2.58109277]
```

The plot of the best fit line



The plot of the cost function vs the number of iterations is given below. We can observe that the cost function decreases with each iteration initially and finally converges after nearly 100 iterations.





Till now we have implemented linear regression from scratch and used gradient descent to find the model parameters. But how good is our model? We need some measure to calculate the accuracy of our model. Let's look at various metrics to evaluate the model we built above.

Evaluating the performance of the model

We will be using Root mean squared error(RMSE) and Coefficient of Determination(R^2 score) to evaluate our model.

RMSE is the square root of the average of the sum of the squares of residuals.

RMSE is defined by

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(x^i) - y^i)^2}$$

```
1 # mean squared error
2 mse = np.sum((y_pred - y_actual)**2)
3
4 # root mean squared error
5 # m is the number of training examples
6 rmse = np.sqrt(mse/m)
```

Rmse.py hosted with ❤ by GitHub

[view raw](#)

RMSE score is **2.764182038967211**.

R^2 score or the **coefficient of determination** explains how much the total variance of the dependent variable can be reduced by using the least square regression.

R^2 is determined by

$$R^2 = 1 - \frac{SS_r}{SS_t}$$

SS_t is the total sum of errors if we take the mean of the observed values as the predicted value.

$$SS_t = \sum_{i=1}^m (y^i - \bar{y})^2$$

SS_r is the sum of the square of residuals

```

1  # sum of square of residuals
2  ssr = np.sum((y_pred - y_actual)**2)
3
4  # total sum of squares
5  sst = np.sum((y_actual - np.mean(y_actual))**2)
6
7  # R2 score
8  r2_score = 1 - (ssr/sst)

```

R2score.py hosted with ❤ by GitHub

[view raw](#)

SS_r - 69.47588572871659
 SS_x - 7.64070234454893
 R^2 score - 0.8900236785122296

If we use the mean of the observed values as the predicted value the variance is 69.47588572871659 and if we use regression the total variance is 7.64070234454893. We reduced the prediction error by ~ 89% by using regression.

Now let's try to implement linear regression using the popular scikit-learn library.

Scikit-learn implementation

scikit-learn is a very powerful library for data-science. The complete code is given below

```

1  # imports
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from sklearn.linear_model import LinearRegression
5  from sklearn.metrics import mean_squared_error, r2_score
6
7  # generate random data-set
8  np.random.seed(0)
9  x = np.random.rand(100, 1)
10 y = 2 + 3 * x + np.random.rand(100, 1)
11
12 # scikit-learn implementation
13
14 # Model initialization
15 regression_model = LinearRegression()
16 # Fit the data(train the model)
17 regression_model.fit(x, y)
18 # Predict
19 y_predicted = regression_model.predict(x)
20
21 # model evaluation
22 rmse = mean_squared_error(y, y_predicted)
23 r2 = r2_score(y, y_predicted)
24

```

```
25 # printing values
26 print('Slope:', regression_model.coef_)
27 print('Intercept:', regression_model.intercept_)
28 print('Root mean squared error: ', rmse)
29 print('R2 score: ', r2)
30
31 # plotting values
32
33 # data points
34 plt.scatter(x, y, s=10)
35 plt.xlabel('x')
36 plt.ylabel('y')
37
38 # predicted values
39 plt.plot(x, y_predicted, color='r')
40 plt.show()
```

Linear Regression using scikit-learn.py hosted with ❤ by GitHub

[view raw](#)

The model parameters and the performance metrics of the model are given below:

```
The coefficient is [[2.93655106]]
The intercept is [2.55808002]
Root mean squared error of the model is 0.07623324582875013.
R-squared score is 0.9038655568672764.
```

This is almost similar to what we achieved when we implemented linear regression from scratch.

That's it for this blog. The complete code can be found in this GitHub repo.

Conclusion

We have learnt about the concepts of linear regression and gradient descent. We implemented the model using scikit-learn library as well.

In the next blog of this series we will take some original data set and build a linear regression model.

[Machine Learning](#) [Linear Regression](#) [Gradient Descent](#)

[About](#) [Help](#) [Legal](#)